ICCAD 2026 CAD Contest Problem D
Timing Fixing by Useful Skew

Aaron Wu
Department of Electrical Engineering
National Taiwan University
b13901011@ntu.edu.tw

Yucheng Liu
Department of Electrical Engineering
National Taiwan University
b13901104@ntu.edu.tw

Sheng-Yu Cheng
Department of Electrical Engineering
National Taiwan University
b13901088@ntu.edu.tw

Hong-Bin Lai
Department of Electrical Engineering
National Taiwan University
b13901078@ntu.edu.tw

Abstract

Timing fixing by useful skew improves circuit timing by modifying the clock tree while keeping data-path logic unchanged. In this work, we formulate ICCAD 2026 CAD Contest Problem D as a constrained local-search problem over legal clock-tree states. Our solver maintains a mutable clock-tree representation and an incremental timing state, enabling reversible apply-score-undo evaluation of local edits, including buffer insertion, inserted-buffer removal, and buffer resizing. To guide the search under undisclosed official weights, we use a proxy objective that balances SS setup repair, FF hold repair, and clock-tree area. We evaluate several candidate-generation policies, ranging from random sampling to violation-guided and union-based timing-aware pools, together with greedy, two-step, simulated annealing, iterated simulated annealing, and tabu-search acceptance strategies. Experiments on five public testcases with five random seeds show that broad candidate diversity and search memory are more effective than narrowly endpoint-focused repair. Among all tested variants, tabu search with random candidates achieves the highest average proxy score, because it improves hold timing strongly while maintaining competitive setup repair and reducing area. These results suggest that useful-skew optimization benefits from balanced timing-area exploration rather than single-component timing maximization.

1 Introduction

Timing closure is a critical task in modern VLSI design. As clock frequency increases, setup and hold violations become harder to fix without disturbing the functional logic of the circuit. ICCAD Contest 2026 Problem D focuses on this setting: given an initial clock tree, a buffer library, and data-path delay reports, the goal is to improve timing quality by modifying the clock tree while preserving the original data-path logic.

The key idea behind this problem is useful skew. Instead of treating all clock skew as harmful, useful-skew optimization intentionally changes clock arrival times at flip-flops to improve timing margins. For a setup-critical path, delaying the capture clock can provide more time for data propagation. For a hold-critical path, delaying the launch clock can postpone data launch and reduce the risk of early data arrival. Therefore, by inserting or resizing buffers in the clock tree, the optimizer can improve setup and hold slack without changing combinational logic.

However, this optimization is highly coupled. A clock-tree edit does not affect only one data path; it changes the clock arrival time of an entire downstream subtree. As a result, a move that improves one violating path may degrade other paths that share the same clock-tree nodes. In addition, buffer insertion and upsizing may increase area, while aggressive area recovery may undo previous timing improvements. The objective is therefore not simply to fix the worst violation, but to balance SS setup improvement, FF hold improvement, and clock-tree area.

In this work, we formulate timing fixing by useful skew as a constrained local-search problem over legal clock-tree states. Our solver maintains a mutable clock-tree representation, evaluates timing and area metrics after local edits, and uses reversible trial operations to explore the search space efficiently. Candidate-generation policies decide which clock-tree edits should be evaluated, while acceptance strategies decide how the solver moves from one tree state to another. The experimental discussion focuses on how these policies trade off setup slack, hold slack, and area.

The rest of this report is organized as follows. Section 2 formulates the timing model, objective, and legal action space. Section 3 describes the local-search framework, including candidate generation, action evaluation, acceptance

strategies, and legality checks. Section 4 reports experimental results and discusses the trade-offs among the tested optimizers. Section 5 concludes the report and outlines future improvements.

2 Problem Formulation

2.1 Input Design and Legal Solution Space

Each testcase consists of an initial clock tree, a buffer library, and two data-path delay reports under the SS and FF corners. The clock tree contains a clock source as the root, clock buffers as internal nodes, and flip-flops as leaf nodes. The buffer library specifies the available buffer types, including their area, maximum fanout constraint, and fanout-dependent delay under each timing corner. The delay reports give the clock period and the fixed data-path delay between launch and capture flip-flops.

The optimizer is allowed to improve timing only by modifying the clock tree. The data-path delay is fixed, so the combinational logic is not part of the optimization space. A legal output must preserve all original contest nodes and their names, and the relative order among original nodes in the clock tree must remain unchanged. Newly inserted buffers must have unique names.

We model a solution as a clock-tree state. A state records the cell type of each buffer, the parent-child connectivity of the clock tree, and the set of buffers inserted by the optimizer. The legal solution space consists of all clock trees reachable from the input tree by legal clock-tree edits while satisfying the structural constraints above.

2.2 Clock Arrival Time and Useful Skew

For each flip-flop, the clock arrival time is the total delay from the clock source to that flip-flop through the clock tree. Since buffer delay depends on the selected cell type, the fanout count, and the timing corner, the clock arrival time is also corner-dependent.

A data path connects a launch flip-flop to a capture flip-flop through combinational logic. Let D_{clk}^{launch} and D_{clk}^{capture} denote the clock arrival times from the clock source to the launch and capture flip-flops. We define clock skew as

$$\text{Skew} = D_{clk}^{\text{capture}} - D_{clk}^{\text{launch}}. \tag{1}$$

With this convention, positive skew means that the capture clock arrives later than the launch clock. A clock-tree edit may change the arrival time of an entire downstream subtree, so the skew of many data paths can change after a single local modification.

2.3 Setup/Hold Slack and Timing Metrics

Let D_{data} be the data-path delay, T_{clk} be the clock period, T_{setup} be the setup requirement, and T_{hold} be the hold requirement. With the skew convention above, the setup and hold slacks are

$$\text{Slack}_{\text{setup}} = T_{\text{clk}} - T_{\text{setup}} - D_{\text{data}} + \text{Skew}, \tag{2}$$

$$\text{Slack}_{\text{hold}} = D_{\text{data}} - T_{\text{hold}} - \text{Skew}. \tag{3}$$

A negative setup slack means that data arrives too late at the capture flip-flop. A negative hold slack means that data arrives too early and may overwrite the previous value. Therefore, a setup violation can often be improved by increasing the capture clock delay, while a hold violation can often be improved by increasing the launch clock delay.

Timing quality is measured using total negative slack (TNS) and worst negative slack (WNS). WNS captures the most severe timing violation among all paths, while TNS measures the accumulated amount of negative slack. In this problem, setup timing is evaluated under the SS corner, and hold timing is evaluated under the FF corner.

2.4 Timing-Area Objective and Internal Proxy Score

The objective combines timing and area. Timing quality is measured by TNS and WNS under the SS and FF corners, while area is the total area of the clock-tree buffers. We define the setup score, hold score, and area score as

$$S_{\text{setup}} = \left(1 - \frac{\text{TNS}_{SS}}{\text{TNS}_{SS}^{ori}}\right) + \left(1 - \frac{\text{WNS}_{SS}}{\text{WNS}_{SS}^{ori}}\right), \quad S_{\text{hold}} = \left(1 - \frac{\text{TNS}_{FF}}{\text{TNS}_{FF}^{ori}}\right) + \left(1 - \frac{\text{WNS}_{FF}}{\text{WNS}_{FF}^{ori}}\right), \quad S_{\text{area}} = 1 - \frac{\text{Area}}{\text{Area}^{ori}}. \tag{4}$$

Each denominator uses the metric value of the original input clock tree. Higher setup and hold scores mean that the negative timing metrics have moved closer to zero, while a higher area score means that the clock-tree buffer area is smaller than the baseline.

The official contest score is a weighted sum of these three components:

$$S_{\text{total}} = \alpha S_{\text{setup}} + \beta S_{\text{hold}} + \gamma S_{\text{area}}. \tag{5}$$

The official weights are not disclosed, so the optimizer cannot directly use the official score as its search objective. According to the Q&A session, the organizers indicated that the weights roughly satisfy $\alpha > \beta \approx \gamma$. Based on this hint, we use a proxy score with fixed weights $\alpha = 0.5$, $\beta = 0.25$, and $\gamma = 0.25$ to guide move selection during optimization. These parameters are not claimed to be the hidden official weights; they are used only as an internal ranking function that reflects the relative importance of setup repair, hold repair, and area control.

This objective captures the main trade-off in the problem. Aggressive buffer insertion or upsizing may improve setup or hold slack, but the same move can reduce the area component and may also hurt timing on other paths. On the other hand, area recovery may remove unnecessary delay but can also undo a previous timing repair. Therefore, the optimizer must balance timing improvement and area cost rather than optimizing only one metric independently.

2.5 Local Edit Semantics

During search, our optimizer explores the solution space with three local edit primitives. First, an *insert* places a new buffer on an existing clock-tree edge, increasing the clock delay of the downstream subtree. This can adjust useful skew, but also increases area.

Second, a *resize* changes the cell type of an existing buffer, providing a finer way to increase or decrease clock delay and area without changing the clock-tree topology. Depending on the selected cell, it may improve timing by adding delay or reduce area by using a smaller buffer.

Third, a *remove* deletes a buffer inserted by the optimizer in an earlier step and reconnects the surrounding tree. This move is used only internally for reversible exploration and area recovery. Original contest nodes remain part of the design and are never removed, so the final output still satisfies the legal modification constraints.

3 Methodology

3.1 Solver Overview

After defining the legal edit operations and the timing-area objective, the remaining question is how to search the large space of legal clock-tree states efficiently. Our solver uses a local-search framework that repeatedly moves from the current clock tree to a nearby legal state. Each iteration is organized around two decisions: which candidate moves should be evaluated, and which evaluated move should be accepted as the next state.

We separate these two decisions into a **candidate-generation policy** and an **acceptance strategy**. The candidate-generation policy defines the local neighborhood visible to the solver in the current iteration. It may focus on violating timing paths, expand around upstream clock-tree regions, or include remove and resize moves for area recovery. The acceptance strategy then decides how the solver moves inside this neighborhood. It may greedily accept the best immediate improvement, probabilistically accept a temporary degradation, or use short-term memory to avoid cycling.

Algorithm 1 summarizes the overall optimization flow. Each iteration first constructs a bounded candidate set, evaluates the candidates through temporary application, and then applies one move according to the acceptance strategy. The solver also maintains the best-seen state separately from the current state, because exploratory strategies may temporarily move to lower-scoring states.

Algorithm 1 Overall local-search optimization flow

Build the initial `ClockTree`, `DataPathGraph`, and `TimingState`
$T_{\text{cur}} \leftarrow$ baseline clock tree
$T_{\text{best}} \leftarrow T_{\text{cur}}, \quad S_{\text{best}} \leftarrow S_{\text{proxy}}(T_{\text{cur}})$
while current time $<$ time budget **do**
    Generate a candidate move set M using the candidate policy
    Evaluate each $m \in M$ by temporary apply-score-undo and collect results in E
    Select a move m^* from E using the acceptance strategy
    if m^* exists **then**
        Apply m^* to T_{cur} and update `TimingState`
        if $S_{\text{proxy}}(T_{\text{cur}}) > S_{\text{best}}$ **then**
            $T_{\text{best}} \leftarrow T_{\text{cur}}, \quad S_{\text{best}} \leftarrow S_{\text{proxy}}(T_{\text{cur}})$
        end if
    end if
end while
Restore T_{best}, perform full timing evaluation, and write `modified_clk_tree.structure`

The rest of this section follows the structure of Algorithm 1. Section 3.2 describes the optimization state and the incremental timing evaluation used inside the temporary apply-score-undo loop. Section 3.3 describes how candidate-generation policies construct the move set M. Section 3.4 describes how acceptance strategies choose the move $m^\star$ from the evaluated set. Finally, Section 3.5 explains how the best-seen internal state is converted into a legal output file.

3.2 Optimization State and Incremental Evaluation

The optimizer maintains three main data structures during search. The `ClockTree` is the mutable solution state and stores the active topology and buffer cell types. The `DataPathGraph` stores the fixed timing paths, including the launch flip-flop, capture flip-flop, and data delay of each path. Since useful-skew optimization modifies only the clock tree, this graph remains read-only. The `TimingState` connects the two structures and caches the quantities needed for fast evaluation, including clock arrivals, setup and hold slacks, TNS, WNS, area, and the proxy score.

Candidate evaluation follows the temporary apply-score-undo loop shown in Algorithm 1. For each trial edit, the solver applies the move to the `ClockTree`, updates the affected clock arrivals and related timing paths in `TimingState`, and computes the proxy-score change. The move is then either accepted or rolled back. Rollback restores both the clock-tree state and the timing cache, so rejected candidates do not affect later evaluations. This design avoids recomputing all clock arrivals and slacks for every trial move. Instead, the solver updates only the parts of the timing state affected by the local edit, allowing many candidate moves to be evaluated within the fixed time budget.

3.3 Candidate Generation

Since the legal edit space is too large to evaluate exhaustively, candidate generation defines the local neighborhood seen by the optimizer at each iteration. A good candidate policy should include moves that are likely to improve useful skew, but it should also keep the candidate set small enough that many iterations can be completed within the time budget. We therefore explored several policies with different degrees of timing awareness and search breadth.

The simplest policy is **random generation**. It uniformly samples legal insert, remove, and resize moves from the current clock tree. This policy is the most naive way to explore the solution space, and it does not directly use timing information to guide move selection. As a result, it may generate many weak moves that do not improve timing. However, it can also expose non-obvious area-recovery or resizing opportunities that more focused policies may miss.

The most direct timing-driven policy is **violation path generation**. It scans all data paths, selects paths with negative slack, and prioritizes the most severe violations. For setup violations, it generates moves near the capture flip-flop; for hold violations, it generates moves near the launch flip-flop. This follows the useful-skew formulas directly, but the resulting neighborhood can be too narrow when the best repair is not close to the violating endpoint.

To address this limitation, the **upstream-window policy** expands the search region from a violating endpoint toward the root of the clock tree. Instead of considering only the endpoint edge, it considers several upstream buffers and edges. These moves may affect a larger downstream subtree and can therefore repair several related timing paths at once.

The **critical-endpoint policy** uses path violations to assign a criticality value to each flip-flop. Negative setup slack contributes to the criticality of the capture flip-flop, while negative hold slack contributes to the criticality of the launch flip-flop. The policy then generates moves around the most critical flip-flops. Compared with selecting individual violating paths, this policy tries to identify endpoints that participate in many violations and may therefore produce moves with broader timing impact.

Finally, the **union-pool policy** combines the action sources of violation path, upstream window, and critical endpoint into one candidate pool. It uses the same total sampling budget as the individual policies, so the goal is not to evaluate more candidates, but to obtain a more diverse candidate set under the same cost. For simulated-annealing-based optimizers, we use a **sampled-union-pool policy**, which samples only one move from this mixed pool in each iteration to match the single-proposal style of simulated annealing.

3.4 Acceptance Strategies

After candidate generation defines the visible neighborhood, the acceptance strategy determines how the search moves through that neighborhood. This decision is important because the proxy score is not a smooth objective. A move that is locally worse may still help the solver reach a better region later, while a purely greedy sequence may quickly become trapped after all immediate improvements are exhausted.

The **greedy strategy** is the most direct baseline. It evaluates all candidates in the current set and applies the move with the largest positive proxy-score improvement. This strategy is stable and easy to interpret: every accepted move immediately improves the current solution according to the proxy score. However, greedy search is also the most vulnerable to local optima.

The **two-step strategy** is designed around the scoring structure of the problem. Since the official objective contains setup, hold, and area components, the first phase focuses on improving setup slack and maximizing Setup Score. The second phase switches back to the full proxy score to recover area and balance setup, hold, and area. This strategy matches the intuition that timing violations should be repaired before area cleanup. Its limitation is that the three components are not independent: area recovery may remove useful delay, while timing-focused edits may introduce area overhead that is difficult to recover later.

Simulated annealing[**?**] is used to handle the non-smoothness of the proxy objective. At each step, it samples a proposal and always accepts improving moves, but it may also accept worsening moves with a temperature-controlled probability. This is useful in our setting because some useful-skew repairs require passing through intermediate states

that temporarily hurt area or another timing corner. The solver records the best-seen state separately, so exploratory moves can be used without making temporary degradation determine the final output.

Iterated simulated annealing extends this idea by alternating between exploration and recovery. Each annealing round explores nearby clock-tree states, and between rounds the solver restores the best-seen state and performs small greedy cleanup batches. This is intended to prevent the current state from drifting too far after many stochastic moves, while still preserving the ability to escape local optima. In this problem, such restoration is important because aggressive timing repairs can accumulate unnecessary inserted buffers and reduce the area component.

Tabu search[**?**] uses short-term memory to guide exploration while keeping each step score-driven. At each iteration, it chooses the best non-tabu move from the evaluated candidate set, and recently used moves are marked tabu to discourage the solver from cycling between similar states. A tabu move can still be accepted by aspiration if it improves the global best score. This is well suited to clock-tree editing because operations can easily undo one another; tabu memory reduces such short cycles while still allowing the search to follow strong proxy-score improvements.

Overall, these acceptance strategies represent different trade-offs between exploitation and exploration. Greedy search tests immediate local improvement, two-step search encodes a timing-first repair schedule, annealing-based methods allow temporary degradation, and tabu search adds memory to avoid cycling. The optimizer variants evaluated in Section 4 combine these acceptance strategies with different candidate-generation policies to study which search behavior best balances setup repair, hold repair, and area control.

3.5 Output Format Validation

Output validity is enforced during move construction rather than repaired after optimization. Inserted buffers receive unique generated names, resize and insertion moves are rejected if the selected cell type violates the fanout limit, and remove moves are limited to optimizer-inserted buffers. Original contest nodes remain active and keep their names, while removed search-only buffers are marked inactive so that they cannot appear in the final output. After the optimizer finishes, the solver restores the best-seen `ClockTree` and writes only active nodes through a preorder traversal with depth information, matching the required clock-tree format.

4 Experimental Results and Discussion

4.1 Score Summary and Experimental Protocol

We evaluate all optimizer variants on five public testcases. Each optimizer is run with five random seeds, resulting in 325 runs in total. We set the optimization time budget to 570 seconds per testcase, leaving a safety margin under the 10-minute contest limit. Table 1 summarizes the final score decomposition of all optimizer variants.

Table 1: Performance comparison of different optimizers. Standard deviation is computed from the final scores over five testcases and five seeds.

Optimizer	Avg. Score	Setup	Hold	Area	Std.
tabu-random	1.140 722	1.276 422	1.916 255	0.093 788	0.235 526
isa-random	1.084 773	1.322 733	1.737 303	−0.043 676	0.273 375
tabu-union-pool	1.062 058	1.259 221	1.731 261	−0.001 474	0.274 410
isa-sampled-union-pool	1.044 759	1.521 136	1.721 173	−0.584 406	0.183 581
sa-sampled-union-pool	1.044 015	1.653 149	1.833 949	−0.964 191	0.165 838
greedy-union-pool	1.042 167	1.193 238	1.757 868	0.024 327	0.266 567
greedy-upstream-window	1.031 489	1.227 970	1.710 767	−0.040 751	0.275 511
greedy-random	1.009 059	1.104 560	1.806 695	0.020 420	0.214 807
sa-random	0.998 808	1.149 864	1.750 100	−0.054 596	0.276 991
two-step-random	0.876 061	1.148 869	1.334 986	−0.128 479	0.318 905
two-step-union-pool	0.859 649	1.045 729	1.405 060	−0.057 920	0.401 216
greedy-violation-path	0.829 324	1.032 678	1.300 464	−0.048 521	0.277 805
greedy-critical-endpoint	0.746 583	0.923 081	1.188 662	−0.048 491	0.375 666

It is noteworthy that *sa-sampled-union-pool* obtains the highest setup score, but its large negative area score reduces its overall ranking. In contrast, *tabu-random* achieves the strongest hold score and maintains a positive area score. To analyze this trade-off more precisely, we use progress traces recorded during each run. Each trace contains elapsed time, best proxy score, SS/FF timing metrics, and clock-tree area. The trajectory figures in the following subsections are generated from these traces. They do not redefine the final ranking in Table 1; instead, they provide a time-resolved decomposition of how each optimizer reaches its final score.

Each trajectory figure contains four panels: best-score gap, setup score, hold score, and area score. For each testcase, the empirical best score is defined as the best score observed across all optimizers, seeds, and time points in our experiments. The best-score-gap panel plots each run relative to this testcase-specific reference, so the gap is at most zero and values closer to zero indicate better anytime performance. This empirical reference is not an optimality certificate; it is only

the best observed score in the traced experiments. The setup, hold, and area panels report the corresponding score components directly. Higher setup and hold scores indicate stronger timing repair, while a higher area score indicates lower clock-tree area relative to the initial tree.

For aggregation, we divide the 570-second runtime into fixed intervals, take the latest recorded value within each interval, and forward-fill missing intervals within the same run. We then compute the median over the five seeds for each testcase and average these testcase-level medians to obtain the plotted curve. The shaded region shows the 25th–75th percentile range across testcases after seed aggregation. Thus, the shaded region represents testcase-to-testcase variation rather than seed-level uncertainty.

The following subsections use these trajectory plots to analyze the mechanisms behind Table 1. We first isolate candidate-generation effects under greedy acceptance, then compare acceptance strategies under fixed candidate families, and finally examine ISA and tabu search as history-based search strategies.

4.2 Effect of Different Candidate Generation Policies

We first isolate the effect of different candidate generation policies by fixing the acceptance policy to greedy best-score selection. Under this setting, the optimizer can only accept an immediately improving move, so the result mainly reflects whether the candidate pool contains useful local edits.

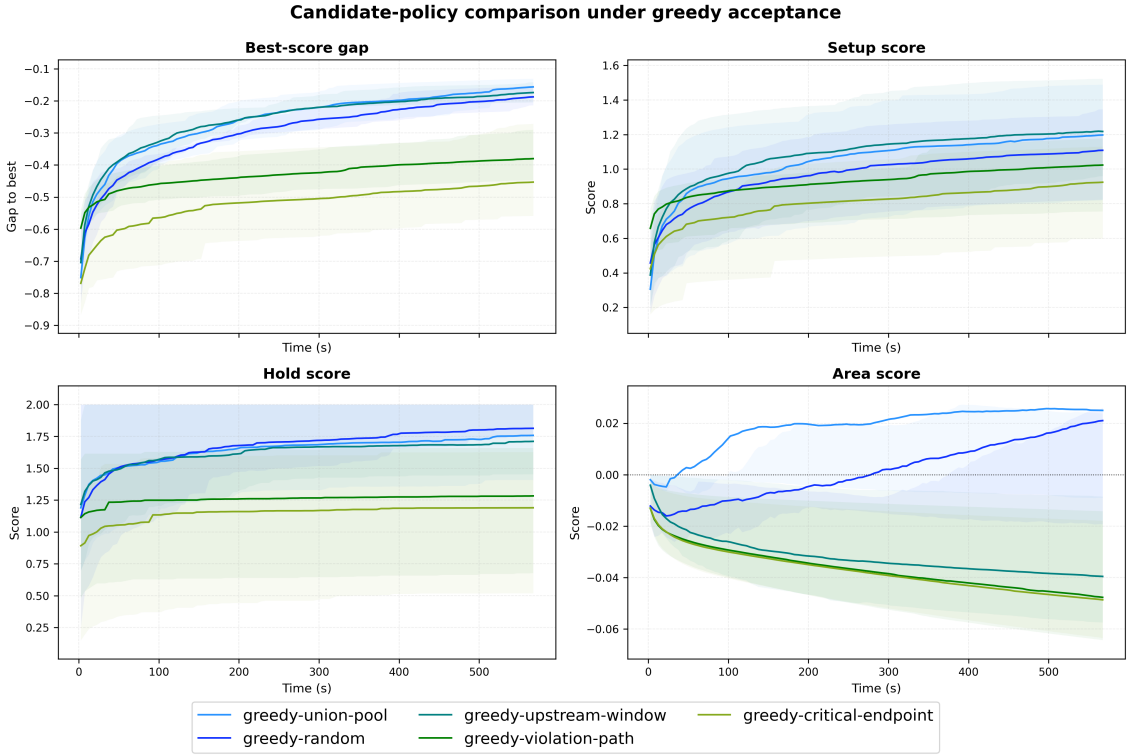

Figure 1: Candidate-policy comparison under greedy acceptance. Each method uses the same greedy acceptance rule, but differs in how candidate clock-tree edits are generated. The panels show the best-score gap, setup score, hold score, and area score over the fixed time budget.

Figure 1 shows that the random, union-pool, and upstream window candidate policies reach the best final best-score gap among the greedy variants, while random remains competitive. The violation-path and critical-endpoint policies improve timing early, but their curves flatten at a worse gap. This supports the claim that endpoint-focused candidate generation is too narrow for useful-skew optimization, because a clock-tree edit affects an entire downstream subtree, and a good repair is often not located directly at the currently violating endpoint. The area panel also explains that pure timing awareness may cause the optimizer to keep many area-expensive edits, while the union-pool and upstream-window and random policies maintain a more balanced area trajectory. Therefore, greedy search benefits more from candidate diversity than from a narrowly targeted violation-only neighborhood.

4.3 Effect of Different Acceptance Policies

We next fix the candidate family and compare how different acceptance policies move through the generated neighborhood. This experiment separates the quality of candidate generation from the ability of the search strategy to escape local optima.

4.3.1 Random Candidate Policy

With random candidates, the optimizer sees a broad but noisy neighborhood. This setting is useful for testing whether an acceptance policy can extract useful moves from an unstructured candidate stream.

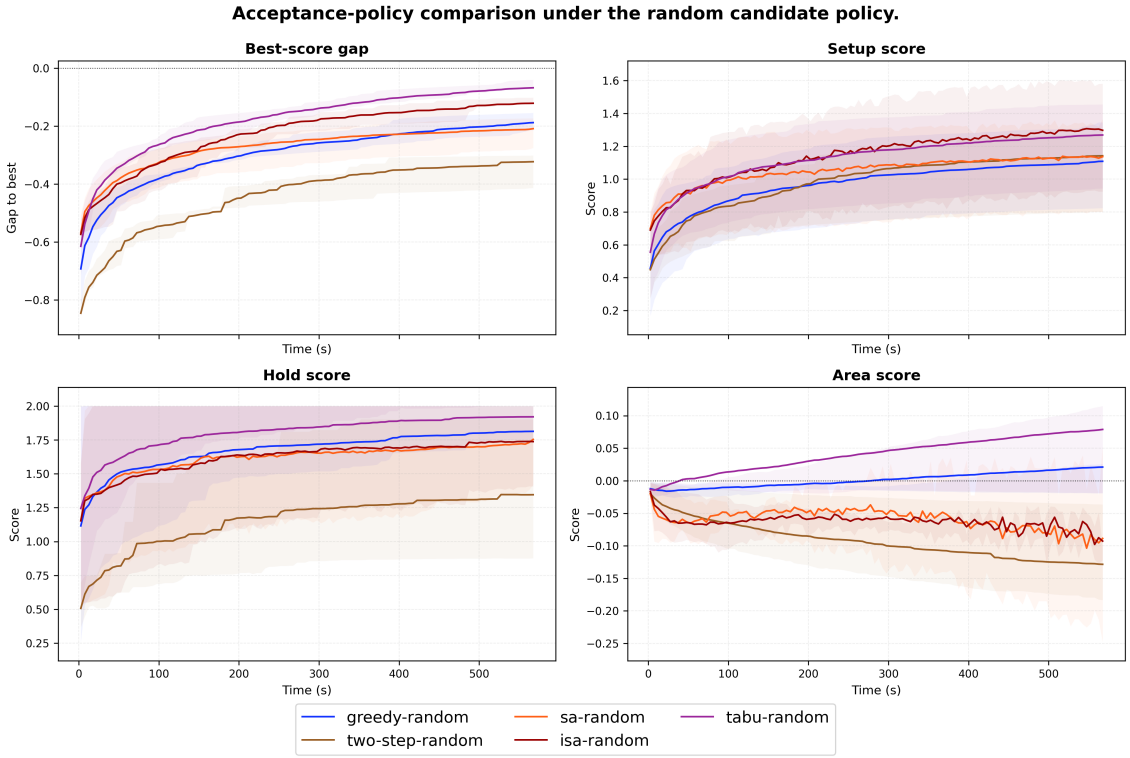

Figure 2: Acceptance-policy comparison under the random candidate policy. The candidate generator is fixed to random sampling, while the acceptance rule varies among greedy, two-step, SA, ISA, and tabu search.

Figure 2 shows that tabu search obtains the closest best-score gap to zero under random candidates. The reason is visible in the component plots: tabu-random is not merely strong in one metric, but maintains a balanced trajectory. It obtains high hold score, competitive setup score, and a positive area score by the end of the run. SA and ISA also escape the limitations of greedy search and improve timing, but their area curves remain negative, indicating that part of their timing gain is paid for by extra buffer area. The two-step method is the weakest in this setting. Its timing-first phase does not create enough durable timing advantage, and the later cleanup phase does not recover enough area or hold quality to close the gap. The main insight is that a random neighborhood can be useful, but only if the acceptance strategy has a mechanism, such as tabu memory, to avoid short cycles while still selecting high-quality moves.

4.3.2 Union-Based Candidate Policies

We also compare acceptance policies when the candidate pool is built from the union of timing-aware sources. For SA and ISA, the sampled-union-pool variant uses a single sampled proposal per iteration to match the single-proposal style of annealing.

Figure 3 shows a different trade-off from the random candidate experiment. *SA-sampled-union-pool* and *ISA-sampled-union-pool* improve setup aggressively and also obtain strong hold scores, but both suffer from a large negative area score, especially SA. This indicates that sampled union proposals are very effective at finding timing-repair moves, but the stochastic acceptance process tends to keep many area-increasing edits. *Tabu-union-pool* is more conservative: its setup score is lower than the annealing-based methods, but its area score stays near the baseline. *Two-step-union-pool* again remains weak because the fixed timing-repair-then-recovery schedule is too rigid for the coupled setup-hold-area objective. Overall, this figure shows that timing-aware candidates are useful for finding repair moves, but the acceptance rule must still control the area cost of these moves.

4.4 ISA and Tabu as Search-Memory Strategies

The previous experiments suggest that the strongest strategies are not the ones that maximize a single component. We therefore compare ISA and tabu search more directly, since both methods use search history to avoid the limitations of plain greedy search. ISA uses repeated annealing rounds with best-state restoration, while tabu search uses a short-term tabu list and aspiration to avoid cycling while still accepting globally promising moves.

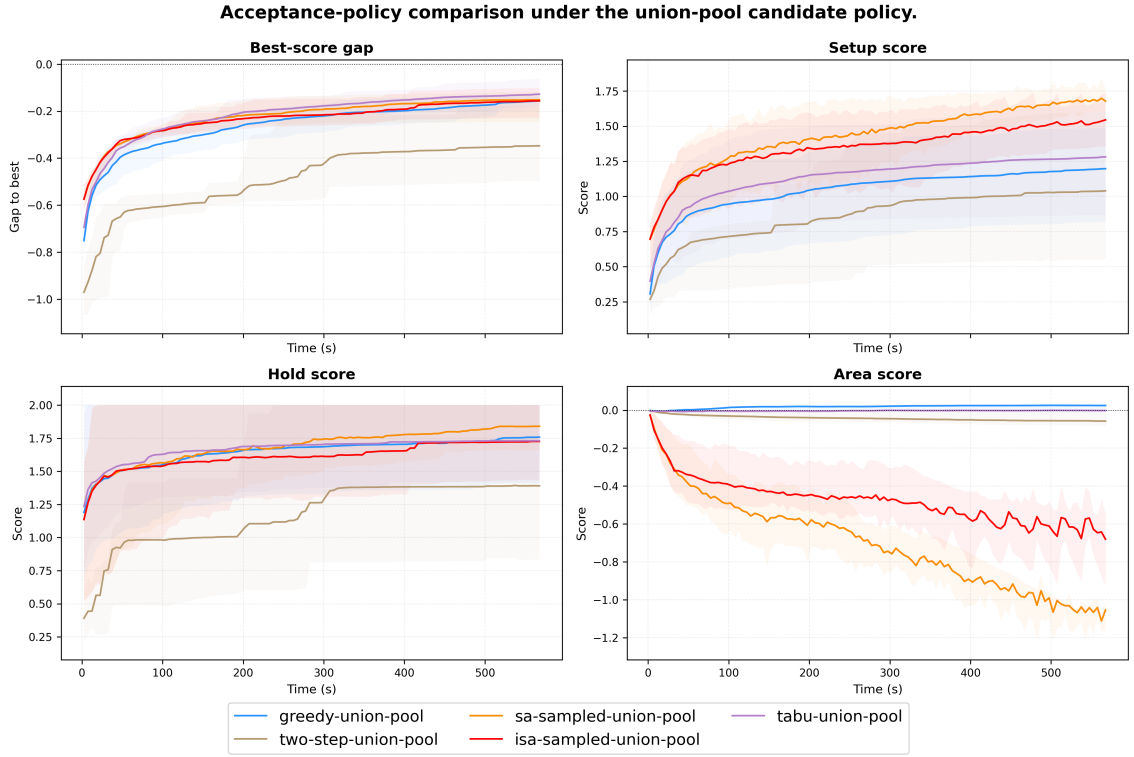

Figure 3: Acceptance-policy comparison under union-based candidate policies. Greedy, two-step, and tabu use the union pool, while SA and ISA use sampled union-pool proposals.

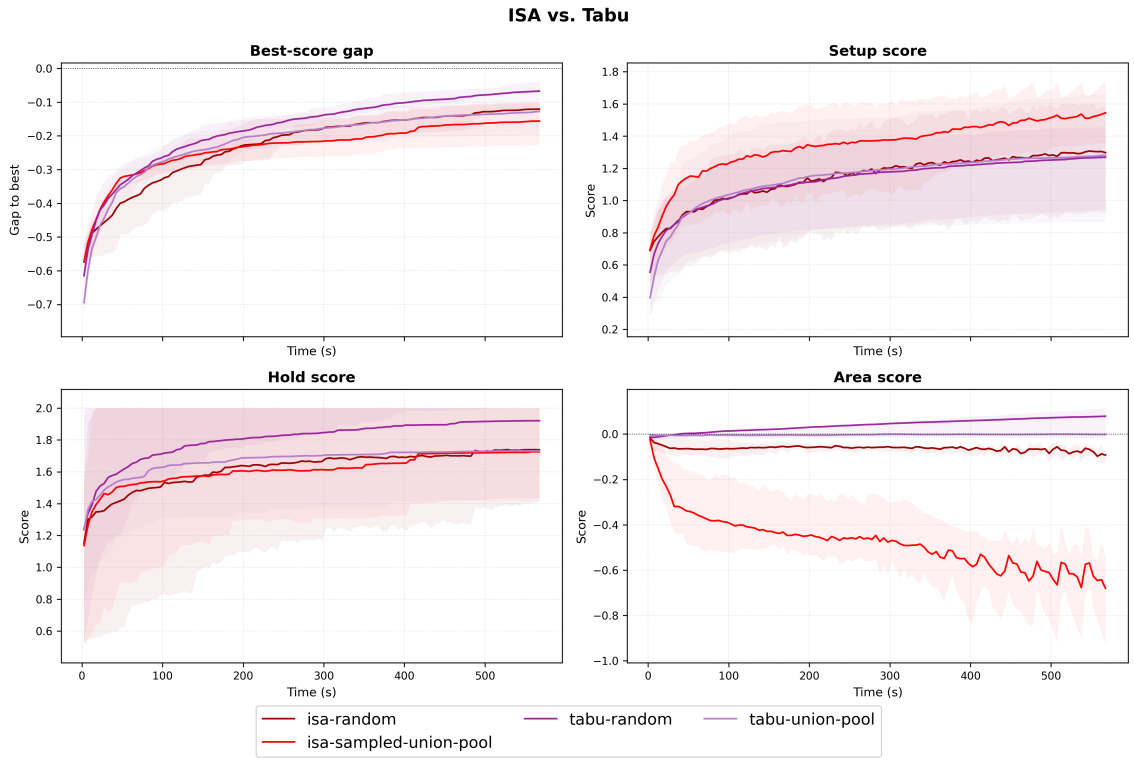

Figure 4: Direct comparison between ISA and tabu variants. The figure compares random and union-based variants of ISA and tabu search using the same four trajectory components.

Figure 4 compares ISA and tabu variants with random and union-based candidates. *ISA-sampled-union-pool* achieves the strongest setup improvement, but this comes with a large area penalty. In contrast, *tabu-random* gives the most balanced trajectory, which it stays close to the empirical best-score gap, improves hold score strongly, and keeps the area score positive. This explains why *tabu-random* ranks highest in the final score table despite not maximizing the setup component.

A noteworthy observation is the interaction between candidate generation and acceptance strategy. For tabu search and ISA, random candidate generation performs better than the corresponding union-based policy. In contrast, under greedy acceptance, the union-pool policy outperforms random generation. This shows that there is no universally best candidate generator; the usefulness of a candidate policy depends on how the search strategy accepts or rejects moves.

For greedy search, the solver can only accept immediately improving moves, so it benefits from a timing-aware candidate pool that provides stronger local improvements. For tabu search and ISA, however, the acceptance strategy already provides exploration through memory, best-state restoration, or temporary degradation. In this setting, random candidates are more useful because they expose a broader and less biased set of insert, remove, and resize opportunities, including area-recovery moves that a timing-focused pool may miss. Therefore, random generation should not be viewed only as a weak baseline. When combined with a memory-based acceptance strategy, it can provide useful candidate diversity and lead to a better setup-hold-area balance.

5 Conclusion

This work addresses timing fixing by useful skew as a constrained local-search problem over legal clock-tree states. Our solver modifies only the clock tree through buffer insertion, inserted-buffer removal, and buffer resizing, while preserving the original data-path logic and output legality. The search is guided by a proxy objective that balances SS setup repair, FF hold repair, and clock-tree area. By combining a mutable clock-tree representation with an incremental timing state, the solver can efficiently evaluate local edits through a reversible apply-score-undo procedure.

The experimental results show that useful-skew optimization depends on both candidate generation and acceptance strategy. Timing-aware candidate pools help greedy search find stronger immediate improvements, but overly targeted candidate generation can miss useful non-local edits. SA and ISA provide stronger exploration and improve setup aggressively, but their area penalty can reduce the final score. Tabu search with random candidates achieves the best average proxy score because it combines broad candidate diversity with score-driven search memory, improving hold timing strongly while maintaining competitive setup repair and positive area score. Overall, the results suggest that this problem is best solved by balancing setup, hold, and area rather than maximizing a single timing component. Future work could further improve the solver by adapting candidate generation to the current violation pattern, calibrating proxy weights more robustly, and adding a stronger late-stage area-aware cleanup phase.

Author Contributions

- **Aaron Wu**: Designed the overall algorithmic framework and implementation plan. Developed and implemented the optimization algorithms, conducted experiments, analyzed the experimental results, and contributed to writing the final report.
- **Yucheng Liu**: Implemented the core data structures and algorithms. Assisted with experimental data analysis, prepared the presentation slides, and contributed to writing the final report.
- **Hong-Bin Lai**: Contributed to the preparation of the presentation slides and the final report.
- **Sheng-Yu Cheng**: Served as a non-enrolled collaborator. Assisted with algorithm design, experimental data analysis, plot generation, report writing, and presentation preparation.

Generative AI Usage Disclosure

We used generative AI tools, including OpenAI Codex, Cursor, and ChatGPT, during this project. These tools assisted with source-code implementation, refactoring, report word refinement, and documentation polishing. Approximately 80% of the submitted source code was initially generated or modified with AI assistance.

The algorithmic ideas, candidate-generation policies, acceptance strategies, experimental design, and final optimizer selection were designed and evaluated by the team. All AI-generated code was manually reviewed, debugged, integrated, and tested. The reported experimental results were obtained by running our implemented solver on the testcases; no experimental data or performance numbers were generated by AI tools.